

Day 11

Managing Users and Permissions in Linux

File Permissions and File Types: explained in simple language

What you will learn	Simple meaning
rwX permissions	Who can read, write or run a file or folder
ls -l	How to read the permission details of a file
chmod	How to change permissions (letters and numbers)
File types	Regular file, directory, link, socket, pipe and device files
file command	How to find out what kind of file something is

Tip: Practice in your own home folder or a practice VM. Do not change permissions of system folders like /etc or /bin.

1. Why File Permissions Are Important

In Linux, many users share one computer. **File permissions** are rules that say **who can read, write, or run** each file and folder. They work like keys: only the right person gets the right access.

Reason	Simple meaning
System Security	Stops unauthorised people from opening your files.
Data Integrity	Only allowed users can change the data, so it stays correct.
Operational Stability	Important system files cannot be changed by mistake, so the system keeps running.

Simple terms you must know

Term	Meaning in simple words
Permission	The right to do something with a file (read, write or run).
Owner	The user who owns the file (usually the one who created it).
Group	A team of users linked to the file.
Others	Everyone else on the system.
Directory	A folder.
Execute	Run a file as a program, or enter a folder.
Symbolic link	A shortcut that points to another file.
Syntax	The correct way to write a command.

2. The rwx Permissions

Letter	Name	On a FILE	On a DIRECTORY (folder)
r	Read	See what is inside the file	List the names of files inside
w	Write	Change the file content	Create, delete or rename files inside
x	Execute	Run the file as a program	Enter the folder (use cd) and reach files in it

Remember: A dash (-) in a permission place means that permission is **not** given. Example: r - - means read only.

Three categories of people

Category	Letter	Who is it?
User (owner)	u	The owner of the file

Category	Letter	Who is it?
Group	g	Users who belong to the group assigned to the file
Others	o	All other users on the system

There is also **a** which means **all** three (u + g + o) together.

3. Reading Permissions with ls -l

Use: `ls` lists files. The option `-l` means long format: it shows full details including permissions.

Syntax: `ls -l [file or folder]`

```
$ ls -l
-rw-r--r-- 1 user group 2048 Dec 18 12:00 file.txt
```

Part	Example	Meaning
1	-rw-r--r--	Permission string (file type + permissions)
2	1	Link count: number of names (links) pointing to this file
3	user	Owner of the file
4	group	Group assigned to the file
5	2048	Size in bytes
6	Dec 18 12:00	Date and time of the last change
7	file.txt	File name

4. Breaking Down the Permission String

Take the example `-rwxr-xr--`. It has 10 characters:

Position	Characters	Meaning
1st	-	File type. (- = regular file, d = directory, l = symbolic link)
2nd to 4th	rwx	Owner can read, write and execute
5th to 7th	r-x	Group can read and execute (no write)
8th to 10th	r--	Others can only read

```
-  rwx  r-x  r--
|  |    |    |
|  |    |    +-- Others: read only
|  |    +----- Group : read + execute
|  +----- Owner : read + write + execute
+----- Type : regular file
```

Numbers for permissions (needed for chmod)

Each permission has a number value. Add the numbers of the permissions you want. One digit is made for each category (owner, group, others).

Permission	Value
r (read)	4
w (write)	2
x (execute)	1
- (none)	0

Letters	Calculation	Number
rwX	4 + 2 + 1	7
rw-	4 + 2 + 0	6
r-x	4 + 0 + 1	5
r--	4 + 0 + 0	4
---	0	0

So `rwXr-xr--` = **7 5 4** = 754, and `rw-r--r--` = 644.

5. The chmod Command

Use: change mode: changes the permissions of a file or folder. Only the owner (or root) can do it.

Syntax (number way): `chmod NUMBER file`

Syntax (letter way): `chmod [u|g|o|a] [+|-|=] [r|w|x] file`

Symbol	Meaning
+	Add a permission
-	Remove a permission
=	Set exactly this permission (replace the old ones)

Practical example 1: owner can read and write, nobody else gets access

```
$ chmod 600 file.txt
$ ls -l
-rw----- 1 user group 1024 Dec 18 14:00 file.txt
```

600 means: owner = 6 (rw-), group = 0 (---), others = 0 (---).

More chmod examples

Command	What it does
<code>chmod 755 file</code>	Owner: rwx, Group: r-x, Others: r-x (common for scripts and programs)
<code>chmod 644 file</code>	Owner: rw-, Group: r--, Others: r-- (common for normal files)
<code>chmod 700 file</code>	Only the owner has full access
<code>chmod +x script.sh</code>	Add execute permission for everyone (a+x)
<code>chmod g+w folder</code>	Give the g roup write permission
<code>chmod o-r file</code>	Remove read from o thers
<code>chmod u=rwx,g=rx,o=r file</code>	Set all three categories in one command
<code>chmod -R 755 folder</code>	Recursive : apply to the folder and everything inside it

Warning: Be careful with `chmod 777`. It gives everyone full access and is a security risk. Avoid it.

6. Introduction to File Types

Linux treats everything as a file. The **first character** of the `ls -l` output tells the file type.

Symbol	File type	Simple meaning
-	Regular file	Normal file with data or text
d	Directory	A folder that holds other files and folders
l	Symbolic link	A shortcut pointing to another file
s	Socket	Lets two running programs talk to each other
p	Pipe	Passes data from one program to another
b	Block device	A hardware device that works in blocks, e.g. hard disk
c	Character device	A hardware device that works one character at a time, e.g. keyboard

```
$ ls -l /dev/sda /dev/tty
brw-rw---- 1 root disk 8,0 Dec 18 10:00 /dev/sda    <- b = block device
crw-rw-rw- 1 root tty  5,0 Dec 18 10:00 /dev/tty    <- c = character device
```

Device files live in the `/dev` folder. (Device names may differ on your machine.)

7. User Defined File Types

These are files that **users create** for their own work:

Type	Meaning	Example
Text files	Notes, configuration, logs or documents	notes.txt

Type	Meaning	Example
Scripts	Files with commands that automate tasks	backup.sh
Executables	Compiled programs (binaries)	a.out

Example 1: create a text file

```
$ echo "Hello, Linux" > myfile.txt
```

echo prints text. The symbol **>** sends that text into a file instead of the screen (it creates the file, or replaces the old content). To check, use `cat myfile.txt`.

Example 2: make a script executable

```
$ chmod +x script.sh
$ ./script.sh          # run it
```

Without **x**, Linux refuses to run the script. **./** means 'in the current folder'.

8. System Defined File Types

These are set up by Linux itself to keep the system working.

Type	Meaning	Location / example
Configuration files	Settings of the system and programs	/etc (e.g. /etc/passwd)
Libraries	Shared code that many programs use	/lib
Logs	Records of what happened on the system	/var/log

Caution: Normal users should only **read** these files. Changing them wrongly can stop the system from working.

9. Practical Scenarios

Scenario 1: Let group members write in a shared folder

```
$ chmod g+w /shared-directory
```

g+w = **group** gets **write**. Members of the folder's group can now create and delete files there. Tip: the group also needs **x** on the folder so they can enter it.

Scenario 2: Keep a file private to the owner

```
$ chmod 700 confidential.txt
$ ls -l confidential.txt
-rwx----- 1 user group 120 Dec 18 15:00 confidential.txt
```

700 = owner rwx, group ---, others ---. (For a plain text file, 600 is enough, because it does not need to run.)

Scenario 3: Find a file's type with the file command

Use: Looks at the file and tells what kind of file it is. **Syntax:** file filename

```
$ file example.txt
example.txt: ASCII text
```

10. Extra: Changing Owner and Group

Sometimes you must change **who owns** a file. Day 10 groups and Day 11 permissions work together here. (This part is an extra, not in your original notes.)

Command	Syntax	Meaning
chown	sudo chown newowner file	Change owner
chgrp	sudo chgrp newgroup file	Change group
chown (both)	sudo chown owner:group file	Change owner and group together

```
sudo chown sourabh:developers report.txt
ls -l report.txt
-rw-r--r-- 1 sourabh developers 500 Dec 18 16:00 report.txt
```

11. Quick Revision Cheat Sheet

Command	What it does
ls -l	Show files with permissions
chmod 600 file	Owner read+write, others nothing
chmod 644 file	Owner read+write, others read only
chmod 755 file	Owner full, others read+execute
chmod 700 file	Only owner has full access
chmod +x file	Make file executable
chmod g+w folder	Give group write permission
chmod -R 755 folder	Apply to folder and everything inside
file name	Show the file type
sudo chown user:group file	Change owner and group

Number	Letters	Number	Letters
7	rwX	3	-wX
6	rw-	2	-w-
5	r-X	1	--X

Number	Letters	Number	Letters
4	r--	0	---

Practice lab (do in order)

1. `cd ~` (go to your home folder)
2. `echo "Hello, Linux" > myfile.txt` (create a file)
3. `ls -l myfile.txt` (see the permissions, usually 644)
4. `chmod 600 myfile.txt` (make it private)
5. `ls -l myfile.txt` (check again: -rw-----)
6. `echo 'echo Hi' > script.sh` (create a script)
7. `./script.sh` (it fails: permission denied)
8. `chmod +x script.sh` (add execute)
9. `./script.sh` (now it prints Hi)
10. `file myfile.txt` (see the file type)

Practice questions

- What do r, w and x mean for a file and for a directory?
- Read this permission string: -rwxr-x---. What can each category do?
- Convert rw-r----- into a number.
- What is the difference between **chmod 600** and **chmod 700**?
- Name any four file types shown by **ls -l** and their symbols.
- Which command tells you the type of a file?

Summary: Conclusion: Permissions decide who can read, write or run a file. Learn to read the permission string, and use chmod carefully to keep your system secure.